

RICERCA SUI PROBLEMI CLASSICI DELLA PROGRAMMAZIONE CONCORRENTE

Sommario

.....	1
PROBLEMI CLASSICI DELLA PROGRAMMAZIONE CONCORRENTE	2
<i>Cos'è la programmazione concorrente.....</i>	<i>2</i>
<i>Concorrenza vs Parallelismo</i>	<i>2</i>
<i>Problemi principali</i>	<i>2</i>
1. Race Condition.....	2
2. Deadlock	2
3. Starvation.....	2
Quattro problemi classici.....	3
1. Problema Produttore-Consumatore.....	3
2. Problema Lettori-Scrittori	3
3. Problema dei Filosofi a Cena	4
4. Problema del Barbiere Sonnolento.....	5

PROBLEMI CLASSICI DELLA PROGRAMMAZIONE CONCORRENTE

Cos'è la programmazione concorrente

La **programmazione concorrente** è un modo di scrivere programmi in cui **più attività vengono eseguite nello stesso periodo di tempo**, invece di una dopo l'altra. Non significa per forza "tutte insieme nello stesso istante", ma **condividendo il tempo della CPU**. Studiando come più processi o thread possano essere eseguiti "allo stesso tempo" condividendo risorse, il problema principale risulta garantire **correttezza**, **coerenza dei dati** e **assenza di deadlock**.

Concorrenza vs Parallelismo

- **Concorrenza**: più attività *in avanzamento nello stesso tempo*
- **Parallelismo**: più attività *eseguite realmente nello stesso istante* (su più core)

Un computer con **1 solo core** può essere concorrente

Un computer con **più core** può essere anche parallelo

Problemi principali

1. Race Condition

Succede quando **più thread accedono agli stessi dati** senza controllo.

Esempio: Due thread aggiornano lo stesso contatore → risultato sbagliato.

2. Deadlock

Due thread si bloccano a vicenda e **nessuno va avanti**.

Esempio:

- Thread A ha risorsa 1 e aspetta la 2
- Thread B ha risorsa 2 e aspetta la 1

3. Starvation

Un thread **non ottiene mai risorse** perché altri hanno sempre la priorità.

Come si risolvono questi problemi

Meccanismi di sincronizzazione per controllare l'accesso alle risorse condivise e consentire ad un solo thread alla volta di accedere alla risorsa:

- **Mutex / Lock**
- **Semafori**
- **Monitor**
- **Sezioni critiche**

Quattro problemi classici

Ora descriviamo alcuni dei problemi classici insieme a schemi esemplificativi e codice Java che mostra una possibile soluzione con meccanismi di sincronizzazione.

1. Problema Produttore-Consumatore

Il Produttore genera elementi e li inserisce in un buffer; il Consumatore li preleva. Il buffer ha capacità limitata e non deve andare né in overflow né in underflow.

Schema [il flusso dei dati e il fatto che il buffer è una risorsa condivisa]

[Produttore] --> [Buffer Limitato] --> [Consumatore]

- **Produttore**: crea dati
- **Buffer**: spazio condiviso dove vengono messi i dati
- **Consumatore**: prende i dati

Il problema

- Se il **buffer è pieno**, il produttore deve fermarsi
- Se il **buffer è vuoto**, il consumatore deve aspettare

```
class Buffer {
    private int data;
    private boolean empty = true;

    public synchronized void produce(int value) throws InterruptedException {
        while (!empty) wait();
        data = value;
        empty = false;
        notifyAll();
    }

    public synchronized int consume() throws InterruptedException {
        while (empty) wait();
        empty = true;
        notifyAll();
        return data;
    }
}
```

2. Problema Lettori-Scrittori

Molti lettori possono accedere contemporaneamente a una risorsa condivisa, ma quando uno scrittore vuole modificare i dati deve avere accesso esclusivo.

Schema [la stessa risorsa viene usata in modi diversi]

Lettori (accesso condiviso)

\ | /
[RISORSA]

/ \
Scrittore (accesso esclusivo)

- Più **lettori** possono leggere **insieme**
- Lo **scrittore** deve essere **da solo**

Il problema

- Se uno scrittore scrive mentre qualcuno legge → dati incoerenti
- Bisogna decidere **chi entra e chi aspetta**

```
import java.util.concurrent.locks.*;

class SharedResource {
    private final ReadWriteLock lock = new ReentrantReadWriteLock();
    private int value = 0;

    public void read() {
        lock.readLock().lock();
        try {
            System.out.println("Lettura: " + value);
        } finally {
            lock.readLock().unlock();
        }
    }

    public void write(int newValue) {
        lock.writeLock().lock();
        try {
            value = newValue;
        } finally {
            lock.writeLock().unlock();
        }
    }
}
```

3. Problema dei Filosofi a Cena

Cinque filosofi siedono attorno a un tavolo. Ognuno ha un piatto di spaghetti ma per mangiare servono **due bacchette**, una a destra e una a sinistra. Se tutti raccolgono una bacchetta contemporaneamente, si crea un deadlock.

Schema [far vedere che le risorse sono condivise e circolari]

[F] - [C] - [F] - [C] - [F] - ...

F = Filosofo, C = Bacchetta

- I filosofi sono seduti in cerchio
- Tra ogni filosofo c'è una **bacchetta condivisa**

Il problema

- Ogni filosofo ha bisogno di **2 bacchette**
- Se tutti ne prendono una sola → **deadlock** (nessuno mangia)

```

class Chopstick {
    private final int id;
    public Chopstick(int id) { this.id = id; }
}

class Philosopher extends Thread {
    private final Chopstick left, right;

    public Philosopher(Chopstick left, Chopstick right) {
        this.left = left;
        this.right = right;
    }

    public void run() {
        try {
            synchronized (left) {
                synchronized (right) {
                    System.out.println("Sto mangiando");
                }
            }
        } catch (Exception ignored) {}
    }
}

```

4. Problema del Barbiere Sonnoletto

Un barbiere ha una sedia per tagliare i capelli e una sala d'attesa con posti limitati. Se non ci sono clienti si addormenta; se arrivano clienti e la sala è piena se ne vanno.

Schema [la gestione dell'attesa e delle risorse limitate]

[**Clienti**] --> [**Sala d'attesa** limitata] --> [**Barbiere**]

- I **clienti** arrivano
- Se c'è **posto** aspettano
- Il **barbiere** lavora un cliente alla volta

Il problema

- Se non ci sono clienti → il barbiere dorme
- Se la sala è piena → i clienti se ne vanno

```

import java.util.concurrent.Semaphore;

class BarberShop {
    private Semaphore customers = new Semaphore(0);
    private Semaphore barber = new Semaphore(0);
    private Semaphore accessSeats;
    private int freeSeats;

    public BarberShop(int seats) {
        this.freeSeats = seats;
        this.accessSeats = new Semaphore(1);
    }

    public void customer() throws InterruptedException {
        accessSeats.acquire();
        if (freeSeats > 0) {
            freeSeats--;
            customers.release();
            accessSeats.release();
            barber.acquire();
        } else {
            accessSeats.release();
        }
    }
}

```

```

public void barber() throws InterruptedException {
    customers.acquire();
    accessSeats.acquire();
    freeSeats++;
    barber.release();
    accessSeats.release();
}

```